Learning Document: Automated Fish Length Estimation Using AutoFish

Purpose of this document. A study guide to understand the project from beginner level and to explain it confidently during the poster presentation and the professor Q&A. It is not just a summary; it walks through the project step by step.

How to read the labels. Throughout, information is tagged so you always know how firmly you can state it:

- ✅ **CONFIRMED FACT** — verified from the code, configs, or saved result files.
- 📊 **RESULT-BASED INTERPRETATION** — a conclusion that follows from our numbers.
- 💡 **HYPOTHESIS** — a possible explanation we did *not* prove; say it as a hypothesis.
- ⚠️ **LIMITATION / UNFINISHED** — something not done or not yet validated.

Authors: Abu Bakar, Laksh Jiwani, Shahman Butt · Supervisor: Bohan Zhuang, M.Sc. · Professor: Stefan Oehmcke · University of Rostock (VACOT). Repository: https://github.com/Shahman-Butt/AreaSeminarFishLength

---

1. Project purpose

Why fish length matters

1. **Simple.** Knowing how long fish are tells scientists how healthy a fish population is: their age, their growth, whether the stock is being overfished, and whether caught fish are above the legal minimum size.
2. **Technical.** Length-frequency distributions feed stock-assessment models in fisheries science; length is a primary biological variable for growth curves, biomass estimation, and quota compliance.
3. **What I did.** We did not collect fish or measure them ourselves; we worked with the AutoFish dataset, where every fish was already hand-measured, and built a system that predicts length from an image.
4. **Why important.** It anchors the whole project in a real scientific need, which is the "motivation" the poster must open with.
5. **On the poster.** Point at the Motivation block: "Fish length is a core measurement in fisheries science, and today it is done by hand."
6. **Q&A.** *Q: Why does fish length matter?* → "It is a key indicator of stock health and legal catch size; automating it saves time and reduces handling of the fish."

Why manual measurement is slow and difficult

1. **Simple.** A person picks up each fish, puts it on a measuring board, reads the length, and writes it down. On a boat with thousands of fish, this is slow, tiring, and stressful for the fish.
2. **Technical.** Manual measurement is labour-intensive, low-throughput, subject to human error and fatigue, and invasive (handling harms the animal).
3. **What I did.** We used images as the input so that, in principle, a camera could replace the board.
4. **Why important.** It explains *why automation is worth attempting* — the gap the project fills.

5. **On the poster.** One short sentence in Motivation: "Manual measuring is slow and invasive."
6. **Q&A.** *Q: Why not just measure by hand?* → "It does not scale, it is error-prone, and it stresses the fish; a camera-based method could measure every fish automatically."

What automated fish length estimation means

1. **Simple.** Show the computer a picture of a fish; it outputs the length in centimetres.
2. **Technical.** A model maps an image (plus, in our case, the fish's bounding box) to a single continuous number, the length in cm.
3. **What I did.** We built exactly this: image in, one length number out.
4. **Why important.** It defines the task type, which drives every design choice.
5. **On the poster.** The workflow diagram shows this: crop → encoder → length.
6. **Q&A.** *Q: What does the system do?* → "It predicts one fish's length in centimetres from a cropped image of that fish."

Detection vs. classification vs. segmentation vs. regression

1. **Simple.**
2. *Detection* = "where is the fish?" (draw a box).
3. *Classification* = "what kind of fish is it?" (pick a category).
4. *Segmentation* = "which exact pixels are the fish?" (trace its outline).
5. *Regression* = "what is the exact number?" (here: how many centimetres).
6. **Technical.** Detection predicts bounding boxes; classification predicts a discrete class; segmentation predicts a per-pixel mask; regression predicts a continuous scalar.
7. **What I did.** Our task is **regression** — predict a continuous length. The dataset already provided boxes and outlines (segmentation), which we *used* as inputs, but we did not train a detector or classifier.
8. **Why important.** Because it is regression, the right error measure is "how many cm off" (MAE), not "accuracy %".
9. **On the poster.** In Data & Setup: "Metric: mean absolute error in cm."
10. **Q&A.** *Q: Is this classification?* → "No. Classification picks a category; we predict a continuous number of centimetres, so it is regression."

Why this project is a regression task

1. **Simple.** The answer is a number that can take any value (30.2 cm, 31.7 cm…), not a fixed set of labels.
2. **Technical.** The target `length_cm` is continuous; the model has a single linear output and is trained with a regression loss (L1). ✅ CONFIRMED FACT (loss `"l1"` in every config).
3. **What I did.** We used an L1 (absolute-error) loss and report MAE in cm.
4. **Why important.** It makes the paper's number (0.62 cm) directly comparable to ours.
5. **On the poster.** The main chart's axis is "MAE (cm)".
6. **Q&A.** *Q: Why regression and not classification?* → "Length is continuous, so we predict a real number and measure the average centimetre error."

The main research question

> **Can modern image encoders and vision foundation models improve fish length estimation compared with the reproduced AutoFish MobileNetV2 baseline?**

The two main tasks

1. **Reproduce the published AutoFish baseline** (MobileNetV2 length regressor) to prove our pipeline is trustworthy.
2. **Replace the image encoder** with DINOv2, CLIP, and ConvNeXt-Tiny and compare fairly, changing *only* the encoder.
3. **Why important.** Task 1 calibrates the "measuring instrument"; Task 2 is the actual research.
4. **Q&A.** *Q: What was the goal?* → "First reproduce the paper's baseline, then test whether newer encoders and foundation models beat it under identical conditions."

---

2. Dataset

The AutoFish dataset (confirmed values)

✅ CONFIRMED FACT (from `data/processed/index.csv` and the build script output): - **1,500 images** - **18,157 fish annotations** - **454 unique fish** - **25 fish groups** - **3,759 test observations** (full test set), of which **1,879 non-occluded** and **1,880 occluded**

1. **Simple.** 1,500 photos of fish on a table; in total the photos contain 18,157 individual fish-appearances, coming from 454 real fish, organised into 25 groups.
2. **Technical.** COCO-style annotations: each record has an image, a species category, a `fish_id`, a hand-measured `length`, a bounding box, and a polygon segmentation.
3. **What I did.** We did **not** create this dataset (it is from Bengtson et al.); we downloaded it from Hugging Face (`vapaau/autofish`) and processed it into one index table.
4. **Why important.** These are the numbers you must quote confidently.
5. **On the poster.** Data & Setup block lists 1,500 / 454 / 18,157 / 25.
6. **Q&A.** *Q: How big is the dataset?* → "1,500 images, 454 unique fish, 18,157 annotations, 25 groups; the test set has 3,759 observations."

Why one physical fish appears in multiple images

1. **Simple.** The same fish was photographed many times, in different arrangements on the table.
2. **Technical.** A `fish_id` recurs across many images/groups; one physical fish generates many annotations.
3. **What I did.** We tracked `fish_id` precisely so the same fish never ends up on both sides of a train/test boundary (see leakage, below).
4. **Why important.** It is exactly why a naive split is dangerous.
5. **On the poster.** Implied by the "no fish appears in two splits" note.

6. **Q&A.** *Q: Why can one fish appear many times?* → "Each fish was photographed repeatedly in different layouts, so it has many annotations."

Test conditions: Set1, Set2, All

✅ CONFIRMED FACT (from `set_name_from_file` in the index builder): - Image numbers 1–20 → **Set1** (non-occluded), 21–40 → **Set2** (non-occluded), 41–60 → **All** (occluded/overlapping). 1. **Simple.** Set1 and Set2 show fish lying separately (easy). "All" shows fish piled up and overlapping (hard). 2. **Technical.** Non-occluded = Set1 ∪ Set2; occluded = All. We report metrics on both regimes plus the full test. 3. **What I did.** Every model is evaluated three ways: non-occluded, occluded, full test. 4. **Why important.** Occlusion is the realistic hard case and separates easy from hard performance. 5. **On the poster.** Mentioned in Data & Setup and in the "occlusion hurts every model" finding. 6. **Q&A.** *Q: What is the difference between the test sets?* → "Set1/Set2 are separated fish; 'All' has overlapping fish, which is harder."

The official group-based split

✅ CONFIRMED FACT (from `OFFICIAL_SPLIT` in the code): - **Train: 15 groups** {2,3,4,5,7,8,9,12,13,15,16,18,19,23,24} - **Validation: 5 groups** {1,6,11,17,25} - **Test: 5 groups** {10,14,20,21,22}

- **Training set** — the fish the model learns from.
- **Validation set** — held-out fish used *during* training to choose the best model version.
- **Test set** — held-out fish used *once* at the end to report the final number.

Why group splitting is necessary / why random image splitting leaks

1. **Simple.** If a fish appears in the training photos *and* the test photos, the model can just recognise that individual fish instead of truly estimating length. That is cheating.
2. **Technical.** Splitting by image would place different images of the same `fish_id` in different splits → identity leakage → optimistic, invalid test scores. Splitting by *group* keeps all appearances of a fish on one side.
3. **What I did.** We adopted the official **group-level** split and verified no `fish_id` crosses splits.
4. **Why important.** It is the single most important fairness safeguard in the project, and a classic exam question.
5. **On the poster.** "Official group-level split. No fish appears in two splits."
6. **Q&A.** *Q: Why group split?* → "Because the same fish appears in many images; a random image split would leak that fish into both train and test and inflate the results."

The `fish_id 113` duplicate

✅ CONFIRMED FACT (from the leakage-audit code and `exclusions.json`): 1. **Simple.** One fish (number 113) accidentally showed up on both sides of the split through a stray duplicate annotation. 2. **Technical.** The audit groups annotations by `fish_id` and counts distinct splits; `fish_id 113` had count > 1. The policy keeps the group(s) where the fish occurs most and drops the low-count duplicate annotation(s); the pipeline then re-checks and hard-fails if any leak remains. 3. **What I did.** We removed that singleton duplicate annotation, logged its `annotation_id` in `exclusions.json`, and re-ran the check to confirm zero leaks. 4. **Why important.** Because of the removal, the **non-occluded test count is 1,879 instead of 1,880** — a small, documented, deliberate difference. It shows we actively guarded against leakage rather than assuming the data was clean. 5. **On the poster.** "(one duplicate fish identity was found and removed)". 6. **Q&A.** *Q: What happened with fish 113?*

→ "It leaked across the split via a duplicate annotation; we removed that one annotation, which is why the non-occluded test count is 1,879, and re-verified there is zero leakage."

---

3. Complete project pipeline

The pipeline, in order. ✅ CONFIRMED FACT that steps 1–15 match the code (`data.py`, `models.py`, `train_baseline.py`, `evaluate.py`); step 7 (bounding box combined with image features) **is confirmed** — see below.

1. **Read the image and annotation.** Load the photo and its record (box, outline, true length).
2. **Locate the fish bounding box.** From the annotation / segmentation mask.
3. **Crop the fish.** Cut a square region around the fish. The pixels outside the fish's outline are blacked out (segmentation masking), so even under occlusion the crop shows one fish.
4. **Resize and normalise.** Resize to 224×224; normalise with ImageNet mean/std. ColorJitter augmentation is applied on the training set only.
5. **Pass the crop through an image encoder.** MobileNetV2, ConvNeXt-Tiny, CLIP, or DINOv2.
6. **Extract image features.** The encoder outputs a feature vector (a list of numbers describing the image).
7. **Combine image features with bounding-box information.** ✅ CONFIRMED FACT: the 4 normalised box values (x/W, y/H, w/W, h/H) are concatenated to the feature vector before the head (`bbox_input: true` in every config; concatenation in `models.py`).
8. **Pass through a regression head.** A small multilayer network (Linear → BatchNorm → ReLU blocks, then a final Linear to 1 output).
9. **Predict one continuous value:** the fish length in cm.
10. **Compare with the true length.** Prediction vs. hand-measured length.
11. **Calculate the training loss.** L1 loss = absolute difference.
12. **Update the trainable parameters.** Adam optimizer adjusts weights via backpropagation.
13. **Select the best checkpoint using validation performance.** Save `best.pt` whenever validation MAE improves.
14. **Evaluate the selected model on the test set.** Once, at the end.
15. **Save metrics and prediction files.** `test_metrics.json`, `history.csv`, and a per-fish `predictions.csv`.

Key terms (connected to this project)

- **Image encoder** — the "eye": turns the fish crop into a feature vector. *This is the only part we swap between experiments.*
- **Feature vector** — the encoder's numeric summary of the image (e.g. 1280 numbers for MobileNetV2, 512 for CLIP, 384 for DINOv2, 768 for ConvNeXt). ✅ CONFIRMED dimensions.
- **Regression head** — the small network that turns (features + box) into the final length number. Head sizes: `[1000,500,1]` for the baseline, `[512,128,1]` for the others. ✅ CONFIRMED.
- **Loss** — how wrong a prediction is; we use L1 (absolute error in cm). ✅ CONFIRMED.
- **Optimizer** — Adam; the algorithm that updates weights to reduce the loss. ✅ CONFIRMED.
- **Learning rate** — step size of each update. Baseline 1e-3; encoder swaps 1e-4; DINOv2 full fine-tune used encoder learning rates 1e-5 and 1e-6. ✅ CONFIRMED.

- **Batch** — how many crops are processed together before one update. Baseline 32; swaps 16. ✅ CONFIRMED.
- **Epoch** — one full pass over the training data. Baseline 200 epochs; swaps 100. ✅ CONFIRMED.
- **Checkpoint** — a saved copy of the model weights (`best.pt` = best on validation, `last.pt` = most recent). ✅ CONFIRMED.
- **Frozen model** — the encoder's weights are locked; only the head trains.
- **Partial fine-tuning** — only the encoder's last block (plus final norm/projection) is unlocked, warm-started from the frozen model's best checkpoint. ✅ CONFIRMED (`resume_checkpoint`).
- **Full fine-tuning** — the whole encoder trains (used for DINOv2 at two learning rates).
- **On the poster.** The workflow strip shows crop → resize → encoder → + box → head → length.
- **Q&A.** *Q: Walk me through the pipeline.* → Use steps 3–9 in one breath: "We crop and mask the fish, resize to 224, run it through the encoder, add the 4 box numbers, and a small head outputs the length in cm."

---

4. Models used

⚠️ Note on causes: we can state *which* model won, but the *reasons* for performance differences are mostly 💡 HYPOTHESES unless labelled otherwise.

MobileNetV2 (the baseline)

1. **Simple.** A small, fast network designed to run on phones.
2. **Technical.** ~3.5M-parameter CNN using depthwise-separable convolutions; ImageNet-pretrained (torchvision V1 weights). ✅ CONFIRMED.
3. **What I did.** Reproduced the AutoFish "REG" baseline: MobileNetV2 fully fine-tuned, head `[1000,500,1]`, L1 loss, Adam 1e-3, batch 32, 200 epochs.
4. **Why important.** It is the reference every other model is compared against, and reproducing it validates the whole pipeline.
5. **On the poster.** Top of the ranking, highlighted.
6. **Q&A.** *Q: Why MobileNetV2?* → "Because the AutoFish paper uses it as its length-regression baseline; we reproduced it before testing anything new."

ConvNeXt-Tiny

1. **Simple.** A modern CNN (2022) built with lessons learned from transformers.
2. **Technical.** ~28M-parameter CNN, ImageNet-pretrained; fully fine-tuned, head `[512,128,1]`, Adam 1e-4, batch 16, 100 epochs. ✅ CONFIRMED.
3. **What I did.** Tested it as a newer supervised encoder after CNNs looked strong.
4. **Why important.** 📊 RESULT-BASED INTERPRETATION: it was the **closest challenger** (0.914 cm), beating both CLIP variants.
5. **On the poster.** Rank 2 on the chart.

6. **Q&A.** *Q: Why ConvNeXt?* → "It is a modern supervised CNN; since supervised encoders looked best, it was the natural next test, and it came closest to the baseline."

CLIP ViT-B/32

1. **Simple.** A model that learned by matching pictures to captions from the internet.
2. **Technical.** OpenCLIP ViT-B/32 with OpenAI weights; 512-d image embedding via `encode_image`. ✅ CONFIRMED. Tested **frozen** and with the **last visual block fine-tuned** (warm-started from the frozen best checkpoint).
3. **What I did.** Two experiments: frozen (1.002 cm) and last-block fine-tuned (0.958 cm).
4. **Why important.** 📊 CLIP transferred **much better than DINOv2**; partial fine-tuning improved it further, but it still did not beat the baseline.
5. **On the poster.** Ranks 3 and 4.
6. **Q&A.** *Q: Why did partial fine-tuning help CLIP?* → "Adapting the last block let CLIP adjust its features to fish while keeping most pretrained knowledge; 💡 our hypothesis is that this avoids overfitting on a small dataset. It improved the result from 1.002 to 0.958 cm."

DINOv2 ViT-S/14

1. **Simple.** A model that taught itself about images with no labels at all.
2. **Technical.** Self-supervised ViT-S/14 (via torch.hub), 384-d CLS-token feature. ✅ CONFIRMED. Tested **frozen**, **last-block fine-tuned**, and **fully fine-tuned** at encoder learning rates 1e-5 and 1e-6.
3. **What I did.** Four experiments (see results).
4. **Why important.** 📊 The **last-block fine-tuned** version was the best DINOv2 (1.439 cm), but every DINOv2 variant trailed the baseline and CLIP.
5. **On the poster.** Ranks 5–8.
6. **Q&A.** *Q: Which DINOv2 was best and why?* → "The last-block fine-tuned one at 1.439 cm. 💡 Hypothesis: adapting only the last block gave enough task-specific change without the instability of full fine-tuning on a small dataset — full fine-tuning at 1e-6 was actually the worst result (2.132 cm)."

---

5. Evaluation metrics

Worked example: suppose a fish is truly **31.0 cm** and the model predicts **30.5 cm**. The error is **−0.5 cm** (predicted minus true).

MAE (Mean Absolute Error) — the main metric

1. **Simple.** On average, how many cm is the prediction off? Ignore the direction of the mistake.
2. **Technical.** MAE = mean(|predicted − true|). ✅ CONFIRMED in `metrics.py`.
3. **What I did.** Report MAE in cm for every model; it is the direct comparison with the paper.
4. **Why important.** Lower MAE is better; **0.771 cm MAE means predictions differ from the true length by about 0.771 cm on average.**

5. **On the poster.** The chart axis.
6. **Q&A.** *Q: What is MAE?* → "The average absolute centimetre error; our best model is 0.771 cm."

RMSE (Root Mean Squared Error)

1. **Simple.** Like MAE, but big mistakes are punished extra hard.
2. **Technical.** RMSE = √mean((predicted − true)^2). ✅ CONFIRMED.
3. **What I did.** Track it alongside MAE (baseline full-test RMSE = 1.268 cm).
4. **Why important.** RMSE ≥ MAE always; a large gap signals some big individual errors.
5. **On the poster.** Not on the main chart (secondary metric).
6. **Q&A.** *Q: Why is RMSE higher than MAE?* → "Because RMSE squares the errors, so a few large mistakes push it above the MAE; the gap tells us occasional predictions are quite far off."

MAPE (Mean Absolute Percentage Error)

1. **Simple.** The error as a percentage of the fish's true length.
2. **Technical.** MAPE = mean(|error| / true) × 100. ✅ CONFIRMED.
3. **What I did.** Baseline full-test MAPE = 2.41%.
4. **Why important.** Puts the cm error in relative terms (about 2.4% of length).
5. **On the poster.** Optional supporting number.
6. **Q&A.** *Q: What does 2.4% MAPE mean?* → "On average the prediction is off by about 2.4% of the fish's length."

Bias

1. **Simple.** Does the model tend to guess too long or too short overall?
2. **Technical.** Bias = mean(predicted − true) — a signed average. ✅ CONFIRMED.
3. **What I did.** Baseline full-test bias = +0.035 cm (essentially none).
4. **Why important.** Near-zero bias means errors are balanced, not systematically high or low.
5. **On the poster.** Optional.
6. **Q&A.** *Q: What does bias mean?* → "Whether predictions are systematically over or under; ours is about +0.035 cm, so basically unbiased."

R^2 (coefficient of determination)

1. **Simple.** How much of the natural variation in fish length the model explains; 1.0 is perfect.
2. **Technical.** $R^2 = 1 - SS_{res}/SS_{tot}$. ✅ CONFIRMED.
3. **What I did.** Baseline full-test R^2 = 0.947.
4. **Why important.** 0.947 means the model explains ~95% of the length variation — a strong fit.
5. **On the poster.** Optional supporting number.
6. **Q&A.** *Q: What does R^2 = 0.95 mean?* → "The model explains about 95% of the variation in fish length across the test set."

---

6. Results

Ranking by full-test MAE ✅ CONFIRMED FACT (from each run's `test_metrics.json`)

Rank	Model	Full-test MAE
1	MobileNetV2 baseline	0.771 cm
2	ConvNeXt-Tiny	0.914 cm
3	CLIP last block fine-tuned	0.958 cm
4	CLIP frozen	1.002 cm
5	DINOv2 last block fine-tuned	1.439 cm
6	DINOv2 frozen	1.738 cm
7	DINOv2 full fine-tuning, LR 1e-5	1.778 cm
8	DINOv2 full fine-tuning, LR 1e-6	2.132 cm

Main findings

- 📊 **MobileNetV2 remained the best model** (0.771 cm).
- 📊 **ConvNeXt-Tiny was the closest challenger** (0.914 cm).
- 📊 **CLIP transferred better than DINOv2** (best CLIP 0.958 vs best DINOv2 1.439 cm).
- 📊 **Partial (last-block) fine-tuning improved both** CLIP (1.002 → 0.958) and DINOv2 (1.738 → 1.439).
- 📊 **None of the tested foundation-model variants beat the baseline.**
- 📊 **Occlusion increased difficulty for every model** (e.g. baseline 0.633 non-occluded → 0.909 occluded).

Baseline reproduction ✅ CONFIRMED

- Paper non-occluded MAE: **0.62 cm**
- Our reproduced non-occluded MAE: **0.633 cm**
- Difference: **0.013 cm ≈ 0.13 mm**

📊 This is a **close reproduction**: a tenth of a millimetre difference is far below any practically meaningful threshold, which validates that our data handling, training, and evaluation are correct.

The occluded discrepancy (be honest)

- Paper occluded MAE: **1.38 cm**
- Our reproduced occluded MAE: **0.909 cm** (ours is *better*)

⚠️ + 💡 **The exact reason is not yet known.** Possible causes are **hypotheses only**: differences in preprocessing, data augmentation, checkpoint selection, other implementation details, our leakage cleanup, or random variation. We do not claim credit for a genuine improvement; it is an open question we would investigate against the official code.

---

7. What I completed ✅ CONFIRMED

For each: what / why / output produced. (We did **not** create the dataset or invent the model architectures — those come from AutoFish and the respective model authors.)

- **Dataset preparation** — parsed the raw annotations into one index table so every step reads the same source of truth. *Output:* `data/processed/index.csv` (18,157 rows).
- **Fish-crop generation** — masked and square-cropped every fish, resized to 224×224, so the model sees one clean fish at a time. *Output:* crop images (one per annotation, 0 missing).
- **Official split implementation** — encoded the 15/5/5 group split in code. *Output:* `splits.json`.
- **Leakage checking** — audited `fish_id` across splits, removed the fish-113 duplicate, and made the check hard-fail on any leak. *Output:* `exclusions.json`, zero-leak guarantee.
- **Baseline reproduction** — trained MobileNetV2 to the paper recipe. *Output:* `runs/baseline_official` (0.633 cm non-occluded).
- **DINOv2 experiments** — frozen, last-block, and two full-fine-tune runs. *Output:* four run folders.
- **CLIP experiments** — frozen and last-block fine-tuned. *Output:* two run folders.
- **ConvNeXt-Tiny experiment** — full fine-tune. *Output:* `runs/convnext_tiny_official`.
- **Server training** — ran all training on a university GPU (NVIDIA RTX 5000 Ada, 32 GB) over SSH.
- **Metric evaluation** — computed MAE/RMSE/MAPE/bias/R^2 on three test subsets per model. *Output:* `test_metrics.json` per run.
- **Result comparison** — assembled the full ranking table.
- **Documentation** — README, full report, summary guide, this learning document, handoff log.
- **Poster-result preparation** — A3 poster with the main MAE chart and presentation prep.

---

8. What I did not complete ⚠️ UNFINISHED

- EfficientNet-B0 experiment
- EfficientNet-B2 experiment
- Multi-seed repetitions
- Confidence intervals
- Statistical significance testing
- Detailed error analysis by species
- Error analysis by fish-length range
- Systematic review of worst predictions
- Detailed occlusion analysis
- Real-time deployment
- Testing on another dataset

Why present findings as initial evidence. Every model was trained **once** (a single fixed seed), so we cannot yet separate a true small difference from run-to-run randomness, and we have no confidence intervals or significance tests. The large gaps (e.g. CNN vs DINOv2) are very unlikely to be noise, but the small gaps (e.g. MobileNetV2 vs ConvNeXt, 0.14 cm) need multi-seed confirmation. So the results are **first experimental findings, not final statistically validated conclusions.**

---

9. Main scientific conclusion

> **Under the tested AutoFish setup, the reproduced MobileNetV2 baseline achieved the lowest full-test MAE. ConvNeXt-Tiny was the closest alternative, CLIP transferred better than the tested DINOv2 configurations, and none of the tested foundation-model variants surpassed the baseline.**

Claims I must NOT make, and why

- ❌ "CNNs are always better than transformers." — We tested a few small models on one dataset; this is not a general law.
- ❌ "Foundation models cannot perform regression." — CLIP reached 0.958 cm; they *can*, just not better than the baseline here.
- ❌ "MobileNetV2 is the best possible model." — We only tested a handful of encoders at small scale.
- ❌ "The ranking is statistically proven." — Single runs, no significance testing.
- ❌ "CLIP definitely understands fish geometry better than DINOv2." — The *why* is a hypothesis; we only observed the numbers.

---

10. Poster presentation guidance

One-minute version

1. **Problem.** Fish length is a key measurement, and doing it by hand is slow and invasive.
2. **Research question.** Can newer encoders or vision foundation models beat the AutoFish MobileNetV2 baseline at estimating fish length?
3. **Method.** We reproduced the baseline, then swapped only the encoder (DINOv2, CLIP, ConvNeXt-Tiny) under identical data, split, and metrics.
4. **Main result.** The baseline stays best at 0.771 cm; ConvNeXt is closest, CLIP beats DINOv2, none beats the baseline.
5. **Limitation and next step.** Single runs so far; next are EfficientNet, multi-seed repeats, and error analysis.

Two-minute version (add)

- **Dataset:** 1,500 images, 454 fish, 18,157 annotations, 25 groups.
- **Group split:** 15/5/5 groups, so no fish leaks between train and test.
- **Baseline reproduction:** 0.633 cm vs the paper's 0.62 cm, so the pipeline is trustworthy.
- **Encoder comparison:** four encoders, eight experiments, identical conditions.
- **Occlusion:** every model does worse on overlapping fish.

Five-minute technical version (add)

- **Preprocessing:** masked square crops at 224×224, ImageNet normalisation, ColorJitter on train.
- **Regression pipeline:** encoder → feature vector → concatenate 4 box values → MLP head → length.

- **Frozen vs fine-tuned:** frozen = head only; partial = last block warm-started; full = whole encoder. Partial helped both CLIP and DINOv2.
- **Metrics:** MAE (main), plus RMSE, MAPE, bias, R^2 (baseline R^2 = 0.947).
- **Full ranking:** read the 8-row table top to bottom.
- **Limitations:** single seed, no significance tests, small FM variants only.

Where to point on the poster

- **Motivation block** — while saying the Problem.
- **Dataset block** — while giving the dataset numbers and the group split.
- **Workflow diagram** — while explaining the Method / pipeline.
- **Baseline reproduction values (the two tiles)** — while saying "0.633 vs 0.62".
- **Main MAE chart** — while giving the Main result and ranking (the focal point).
- **Take-home message box** — while delivering the conclusion sentence.
- **Limitations and future work** — while saying the Limitation and next step.

---

11. Important poster questions

Each answer has: **Direct answer · Supporting result · Limitation/uncertainty · Short version (if nervous).**

1. What was the main goal? Direct: Reproduce the AutoFish MobileNetV2 baseline, then test whether newer encoders or foundation models beat it. Support: baseline reproduced at 0.633 cm. Limitation: only a handful of encoders tested. Short: "Reproduce the baseline, then fairly compare newer encoders."

2. What exactly was predicted? Direct: One number, the fish's length in centimetres, from a cropped image plus its bounding box. Support: L1 regression, MAE in cm. Limitation: one fish per crop, not whole scenes. Short: "The length of one fish in cm."

3. Why is it a regression task? Direct: The target is a continuous number, not a category. Support: L1 loss, MAE metric. Limitation: none. Short: "Because length is a continuous value."

4. Why reproduce the baseline first? Direct: To prove our pipeline is correct before comparing anything. Support: 0.633 vs 0.62 cm. Limitation: reproduction is close on non-occluded but differs on occluded. Short: "To trust our own setup before comparing models."

5. Was the reproduction successful? Direct: Yes on non-occluded fish, within 0.013 cm of the paper. Support: 0.633 vs 0.62. Limitation: occluded differs (0.909 vs 1.38), reason unknown. Short: "Yes, within a tenth of a millimetre on non-occluded fish."

6. Why is the occluded result different from the paper? Direct: We don't know for certain; it is an open question. Support: ours 0.909 vs paper 1.38. Limitation/uncertainty: 💡 possible causes are preprocessing, augmentation, checkpoint selection, implementation, our leakage cleanup, or randomness — all hypotheses. Short: "We're not sure yet; it is on our list to check against the official code."

7. Why a group-level split? Direct: So the same fish never appears in both training and test. Support: zero leakage after the audit. Limitation: fewer groups means less data per split. Short: "So no fish leaks between train and test."

8. What is data leakage? Direct: When test information reaches training, making scores look better than reality. Support: here it would be the same fish in both splits. Limitation: we only checked identity leakage. Short: "Test info sneaking into training and faking good results."

9. What happened with fish ID 113? Direct: It appeared on both sides of the split via a duplicate annotation; we removed that one annotation. Support: non-occluded test count is 1,879 not 1,880. Limitation: a single-sample change, negligible on metrics. Short: "A leaked duplicate we removed; that's why the count is 1,879."

10. What information goes into the model? Direct: The masked, resized fish crop plus 4 normalised bounding-box numbers. Support: `bbox_input: true` in every config. Limitation: no other metadata (species, camera) is used. Short: "The fish crop and its bounding box."

11. What is the role of the bounding box? Direct: It restores scale information lost when the crop is resized. Support: used by the original paper baseline too. Limitation: it is a strong prior. Short: "It tells the model the real size of the crop."

12. Could the bounding box alone predict length? Direct: It is a strong cue, but not sufficient — all models share the same box yet differ by up to 1.4 cm, so image features matter. Support: identical box input, different results. Limitation: 💡 we did not run a box-only ablation. Short: "It helps a lot, but the image still matters; the box alone wasn't tested in isolation."

13. Why test DINOv2 and CLIP? Direct: They are leading vision foundation models; the question is whether their general features help this precise task. Support: both tested under identical conditions. Limitation: only small variants (ViT-S/14, ViT-B/32). Short: "They are the big general-purpose vision models we wanted to test fairly."

14. What does a frozen encoder mean? Direct: The encoder's weights are locked; only the small head trains. Support: frozen CLIP 1.002, frozen DINOv2 1.738. Limitation: frozen may under-use the encoder. Short: "The encoder isn't trained; only the head learns."

15. Why did partial fine-tuning help? Direct: Adapting only the last block lets the encoder adjust to fish while keeping most pretrained knowledge. Support: CLIP 1.002 → 0.958, DINOv2 1.738 → 1.439. Limitation: 💡 the mechanism is a hypothesis (less overfitting than full fine-tune on small data). Short: "It adapts a little to fish without forgetting everything else."

16. Why did DINOv2 perform poorly? Direct: All its variants trailed the baseline. Support: best DINOv2 = 1.439 cm. Uncertainty: 💡 hypothesis — its self-supervised CLS features favour "what is this" over precise size/geometry, and full ViT fine-tuning on ~11k crops was unstable (1e-6 was worst at 2.132). Short: "Its features seem tuned for recognition, not precise measurement — but that's a hypothesis."

17. Why did CLIP beat DINOv2? Direct: Under identical conditions CLIP was far better (0.958 vs 1.439). Uncertainty: 💡 hypothesis — CLIP's large image-text pretraining may retain more size/shape cues. Short: "CLIP transferred much better; we think its image-text training keeps more shape information, but that's unproven."

18. Why did ConvNeXt-Tiny come close to MobileNetV2? Direct: Both are supervised ImageNet CNNs, fully fine-tuned. Support: ConvNeXt 0.914 vs baseline 0.771. Uncertainty: 💡 supervised CNN features may suit this task; also ConvNeXt used a different recipe (1e-4/100 epochs) than the baseline (1e-3/200), so part of the gap could be recipe not architecture. Short: "It's a modern supervised CNN; it came second, though recipe differences matter."

19. Why did MobileNetV2 remain best? Direct: The fully task-adapted lightweight CNN gave the lowest MAE (0.771). Uncertainty: 💡 task adaptation seems to beat general features here; single-run result. Short: "A small

network trained end-to-end for this exact task won."

20. Was the comparison completely fair? Direct: Very controlled — same data, split, input, head, loss, metrics, and a shared training budget for the swaps. Limitation: the baseline used its own paper recipe, and we did no per-encoder tuning, which could under-serve the challengers. Short: "As fair as possible; only the encoder changed, though we didn't tune each one separately."

21. Why keep the same split and metrics? Direct: So any difference is due to the encoder, not the setup, and stays comparable to the paper. Support: identical split/metrics everywhere. Limitation: none. Short: "So the comparison is apples-to-apples."

22. What does 0.771 cm MAE mean? Direct: On average the prediction is about 0.771 cm from the true length. Support: full-test, 3,759 fish. Limitation: average hides individual large errors (RMSE 1.268). Short: "We're off by about 0.77 cm on average."

23. Why is RMSE higher than MAE? Direct: RMSE squares errors, so large mistakes weigh more. Support: baseline RMSE 1.268 vs MAE 0.771. Limitation: indicates some big individual errors. Short: "Because RMSE punishes big errors harder."

24. What does bias mean? Direct: Whether the model over- or under-predicts on average. Support: baseline bias +0.035 cm (basically none). Limitation: near-zero overall can still hide subgroup bias. Short: "Systematic over/under-prediction; ours is almost zero."

25. What does R^2 mean? Direct: The fraction of length variation the model explains. Support: baseline R^2 = 0.947. Limitation: high R^2 still allows ~0.77 cm error. Short: "How much of the variation we explain — about 95%."

26. How did occlusion affect performance? Direct: Every model did worse on overlapping fish. Support: baseline 0.633 → 0.909 cm. Uncertainty: no detailed occlusion analysis yet. Short: "Overlapping fish are harder for every model."

27. Why not multiple seeds? Direct: Time and GPU constraints; it is the planned next step. Limitation: without it we can't test significance of small gaps. Short: "Not done yet — it's the next step."

28. How reliable is the ranking? Direct: The large gaps are reliable; the small ones need multi-seed confirmation. Support: 0.7 cm CNN-vs-DINOv2 gap vs 0.14 cm baseline-vs-ConvNeXt gap. Limitation: single runs. Short: "Big differences are solid; small ones need repeats."

29. What was your personal contribution? Direct: All the engineering and experiments — data processing, crops, split, leakage audit, baseline reproduction, all eight encoder runs, evaluation, comparison, and documentation. Limitation: dataset and model architectures are not ours. Short: "We built and ran the whole pipeline and all experiments."

30. What came from the AutoFish paper? Direct: The dataset, the baseline method (MobileNetV2 regressor), the official split, and the target numbers. Limitation: we reproduced, not re-invented, the baseline. Short: "The data, the baseline idea, and the split."

31. What surprised you most? Direct: How much better CLIP transferred than DINOv2 under identical conditions. Support: 0.958 vs 1.439. Uncertainty: the reason is a hypothesis. Short: "CLIP beating DINOv2 by so much."

32. What is the biggest limitation? Direct: Single training runs with no significance testing. Support: one seed per model. Short: "Only one run per model so far."

33. What would you do next? Direct: EfficientNet-B0, then multi-seed repeats and error analysis. Short: "EfficientNet, repeats with more seeds, and error analysis."

34. Why EfficientNet-B0 next? Direct: Supervised CNNs (MobileNetV2, ConvNeXt) look strongest, and EfficientNet is the next natural supervised family to test. Short: "Because supervised CNNs are winning, and it's the next one to try."

35. Can the system detect fish automatically? Direct: Not in our setup — we use the dataset's provided boxes/masks; detection was out of scope. Limitation: a full system would need a detector first. Short: "No, we assume the fish is already located."

36. Can it generalise to new datasets or cameras? Direct: Unknown — we only tested on AutoFish. 💡 It would likely need retraining or adaptation. Limitation: no cross-dataset test done. Short: "We haven't tested that; probably not without retraining."

37. Can it work in real time? Direct: Not tested; MobileNetV2 is lightweight so it is plausible. 💡 Hypothesis, not measured. Short: "Not tested, but the small model makes it plausible."

38. Why is the project useful even though the baseline won? Direct: A negative result is still knowledge — it shows foundation models are not automatically better for precise regression, and it validates a trustworthy baseline. Short: "It shows newer isn't automatically better, which is a useful finding."

39. What is the main take-home message? Direct: For precise fish length regression on AutoFish, a small task-adapted CNN beat the tested foundation models. Short: "Task-adapted CNNs beat the foundation models we tested."

40. Which conclusion are you most confident about? Direct: That the baseline was faithfully reproduced (0.633 vs 0.62 cm) and that DINOv2 (as tested) transfers poorly to this task — both are large, clear effects. Short: "The reproduction is solid, and DINOv2 clearly underperformed."

Final revision materials

1. One-page revision sheet

- **Goal:** reproduce AutoFish MobileNetV2 baseline, then fairly compare newer encoders.
- **Task type:** regression (predict length in cm), main metric MAE.
- **Data:** 1,500 images · 454 fish · 18,157 annotations · 25 groups · test 3,759 (1,879 non-occ / 1,880 occ).
- **Split:** group-level 15/5/5; avoids fish leakage; fish-113 duplicate removed → 1,879.
- **Pipeline:** masked square crop → 224×224 → encoder → features + 4 box values → MLP head → length.
- **Models:** MobileNetV2 (baseline), ConvNeXt-Tiny, CLIP ViT-B/32 (frozen/last-block), DINOv2 ViT-S/14 (frozen/last-block/full×2).
- **Result:** baseline best (0.771 full-test MAE); ConvNeXt 0.914; CLIP 0.958/1.002; DINOv2 1.439+.
- **Reproduction:** 0.633 vs paper 0.62 (non-occluded); occluded 0.909 vs 1.38 (reason unknown).
- **Conclusion:** no tested foundation model beat the baseline; findings are initial (single-seed).
- **Next:** EfficientNet-B0, multi-seed, error analysis.

2. Ten numbers to memorise

1. **0.771 cm** — best full-test MAE (MobileNetV2).
2. **0.633 cm** — our non-occluded reproduction.
3. **0.62 cm** — paper non-occluded baseline.
4. **0.909 cm** vs **1.38 cm** — our vs paper occluded.
5. **0.914 cm** — ConvNeXt-Tiny (2nd).
6. **0.958 / 1.002 cm** — CLIP fine-tuned / frozen.
7. **1.439 cm** — best DINOv2 (last-block).
8. **1,500 / 454 / 18,157 / 25** — images / fish / annotations / groups.
9. **15 / 5 / 5** — train / val / test groups; test n = **3,759** (1,879 / 1,880).
10. **R^2 0.947**, RMSE **1.268**, MAPE **2.41%**, bias **+0.035** — baseline full-test extras.

3. Ten technical terms to understand

Regression · Image encoder · Feature vector · Regression head · Bounding box (bbox input) · MAE · Group-level split · Data leakage · Frozen vs partial vs full fine-tuning · Checkpoint selection on validation.

4. Five conclusions I can safely defend

1. The MobileNetV2 baseline was faithfully reproduced on non-occluded fish (0.633 vs 0.62 cm).
2. On this setup, no tested foundation-model variant beat the baseline.
3. CLIP transferred clearly better than the tested DINOv2 configurations.
4. Partial (last-block) fine-tuning improved both CLIP and DINOv2 over their frozen versions.
5. Occlusion made the task harder for every model.

5. Five claims I must avoid

1. "CNNs are always better than transformers."
2. "Foundation models cannot do regression."
3. "MobileNetV2 is the best possible model."
4. "The ranking is statistically proven."
5. "CLIP understands fish geometry better than DINOv2" (stated as fact rather than hypothesis).

6. Final one-minute poster script

> "Fish length is a key measurement in fisheries science, and measuring by hand is slow and invasive. We asked whether modern encoders or vision foundation models can beat the AutoFish MobileNetV2 baseline at estimating fish length. First we reproduced that baseline, matching the paper within about a tenth of a millimetre, which tells us our pipeline is trustworthy. Then we swapped only the image encoder, keeping the data, split, and metrics identical, and tested ConvNeXt-Tiny, CLIP, and DINOv2. The baseline stayed best at 0.771 cm average error; ConvNeXt was closest, CLIP transferred better than DINOv2, and none of the foundation models beat the baseline. So for this precise measurement task, a small task-adapted network still wins. These are initial single-run findings, so our next steps are EfficientNet, multi-seed repeats, and a detailed error analysis."